Local Jarvis Architecture

This architecture uses a local-first assistant with three Ollama-backed models: a general assistant model, a coding model, and a lightweight router model.[web:854][web:866][web:878] It combines UI, orchestration, memory, tools, coding-agent workflows, and optional cloud fallback in one modular design so each task can be routed to the right subsystem.[web:814][web:821][web:849]

Full diagram

```
flowchart TD
    U[User]

    subgraph I[Interface Layer]
        UI1[Streamlit Chat UI]
        UI2[Voice Input / STT]
        UI3[Voice Output / TTS]
        UI4[File Upload / Image Input]
    end

    subgraph G[Gateway Layer]
        API[FastAPI Backend]
        AUTH[Session / Auth / Rate Guard]
        VAL1[Input Guardrails]
        VAL2[Output Guardrails / PII Redaction]
        OBS[Tracing / Logs / Metrics]
    end

    subgraph O[Orchestration Layer]
        ORCH[LangGraph Orchestrator]
        ROUTER[Task Router]
        PLAN[Planner]
        MEMCTX[Context Builder]
        HITL[Human Approval Check]
    end

    subgraph M[Model Layer via Ollama]
        RMODEL[Router Model\nphi3:mini]
        GMODEL[General Assistant Model\nllama3.2:8b]
        CMODEL[Coding Model\nqwen2.5-coder:7b]
        FALLBACK[Optional Cloud Model Fallback]
    end

    subgraph K[Memory and Knowledge Layer]
        SMEM[Short-term Conversation Memory]
        LMEM[Long-term User Memory]
        VDB[Vector DB / Chroma]
        DOCS[Local Documents / Notes / PDFs]
        KBIDX[Embedding + Chunking Pipeline]
    end
```

```
    subgraph T[Tool Layer]
        T1[Weather Tool]
        T2[Calculator Tool]
        T3[Document Search Tool]
        T4[Calendar / Reminder Tool]
        T5[System Automation Tool]
        T6[Web Search Tool]
        T7[Email / Notification Tool]
    end

    subgraph C[Coding Agent Layer]
        CA1[Repo Scanner]
        CA2[Read File Tool]
        CA3[Search Code Tool]
        CA4[Patch / Write File Tool]
        CA5[Run Tests Tool]
        CA6[Run Terminal Command Tool]
        CA7[Explain Error Tool]
        CA8[Code Review / Refactor Tool]
    end

    subgraph S[Safety and Policy Layer]
        P1[Prompt Injection Check]
        P2[Permission Policy]
        P3[Risk Classifier]
        P4[Action Confirmation Gate]
    end

    U --> UI1
    U --> UI2
    U --> UI4

    UI1 --> API
    UI2 --> API
    UI4 --> API
    API --> AUTH
    AUTH --> VAL1
    VAL1 --> P1
    P1 --> ORCH
    API --> OBS

    ORCH --> ROUTER
    ORCH --> PLAN
    ORCH --> MEMCTX
    ORCH --> HITL
    MEMCTX --> SMEM
    MEMCTX --> LMEM
    MEMCTX --> VDB

    DOCS --> KBIDX
    KBIDX --> VDB

    ROUTER --> RMODEL
    RMODEL --> GMODEL
    RMODEL --> CMODEL
```

```
    RMODEL --> FALLBACK

    PLAN --> GMODEL
    PLAN --> CMODEL

    GMODEL --> T1
    GMODEL --> T2
    GMODEL --> T3
    GMODEL --> T4
    GMODEL --> T5
    GMODEL --> T6
    GMODEL --> T7

    T3 --> VDB
    T3 --> DOCS

    CMODEL --> CA1
    CMODEL --> CA2
    CMODEL --> CA3
    CMODEL --> CA4
    CMODEL --> CA5
    CMODEL --> CA6
    CMODEL --> CA7
    CMODEL --> CA8

    CA1 --> CA2
    CA1 --> CA3
    CA3 --> CA2
    CA4 --> P2
    CA5 --> CA6
    CA6 --> P3
    P3 --> P4
    P2 --> HITL
    P4 --> HITL

    HITL --> VAL2
    GMODEL --> VAL2
    CMODEL --> VAL2
    FALLBACK --> VAL2
    VAL2 --> UI1
    VAL2 --> UI3
    VAL2 --> OBS
```

Main flow

1. The user talks through chat, voice, or files into the interface layer.
2. FastAPI receives the request, applies guardrails, and passes it to LangGraph orchestration.
3. The router decides whether the task is general-assistant work, coding-agent work, or a fallback case.[web:816][web:878]
4. The chosen model uses memory and tools, and sensitive actions go through permission and approval checks before the final answer is returned.[web:821][web:849]

Model routing rules

- `phi3:mini` handles lightweight intent classification and query routing.[web:879][web:884]
- `llama3.2:8b` handles general chat, planning, memory-aware conversation, retrieval, and normal assistant tasks.[web:878]
- `qwen2.5-coder:7b` handles repo analysis, code generation, debugging, patching, testing, and terminal-style coding workflows.[web:878][web:881]
- An optional cloud model is only used when the local models fail or when the task exceeds local capability.[web:854][web:866]

Coding-agent path

The Claude-Code-like part is the coding-agent layer. It should be separated from the general assistant so coding tasks can use repo-specific tools such as file reading, code search, patch writing, test running, and terminal commands without polluting normal chat flows.[web:859][web:865]
Sensitive write or execute actions should pass through permission policy, risk classification, and approval gates before changes are applied.[web:821][web:849]